Hashing

Problem 1.

For each of the proposed hash functions given, determine whether they can be implemented and used as actual hash functions for a hash table with N slots.

(a) $h(k) = 0$

(b) $h(k) = random(0, N)$ (where $random(0, N)$ gives a uniformly generated random number from 0 to $N - 1$).

(c) $h(k) = k \mod N$.

(d) $h(k) = k \mod (N - 2)$ (Assume $N \geq 3$).

(e) $h(k) = f(k)$ (where $f(N)$ is a deterministic function that runs in $\Theta(N^2)$ time and returns a value from 0 to $N - 1$).

Solution

Answers: a),c),d),e): Yes, b): No. Justification for e): Even though (a) will always result in collisions, it is still a valid hash function. For (b), because the same input may not give the same output has, it cannot be used as a valid hash function. For (e), it can be used as a hash function even though it has a bad runtime as it satisfies the key idea of a hash function, that it maps arbitrary key values to a fixed set of integers.

Problem 2.

Consider the hash function $h(k) = k \ mod \ m$, where $m = 2^p - 1$ and k is a character string interpreted in radix (base) 2^p . Show that if we can derive string x from string y by permuting its characters, then x and y hash to the same value.

Solution

$h(x) = (\sum_{i=0}^{n-1} x_i 2^{ip}) \, mod \, m = (\sum_{i=0}^{n-1} x_i 2^{ip}) \, mod \, (2^p - 1)$. Observe that $2^{ip} \, mod \, (2^p - 1) = (2^p)^i \, mod \, (2^p - 1) = 1$ since $2^p \, mod \, (2^p - 1) = 1$. Hence the sum is equal to $(\sum_{i=0}^{n-1} x_i) 1 \mod (2^p - 1)$ and does not depend on the order of the elements in the sum.

Problem 3.

Comparing open addressing to traditional chaining implementations it is true that under the same loading factor:

(a) In open addressing it takes on the average less time to do an unsuccessful search.
Answer: Unsuccessful search takes $1/(1-\alpha)$ steps on the average whereas using chaining it takes a steps on the average. Since we require (because of open addressing) $\alpha < 1$, open addressing is worse.

(b) In open addressing insertions and deletions are faster.
Answer: Since insertions take about the same time as unsuccessful searches, then open addressing must again be worse. Also deletions could be done in one step if we keep pointers.

(c) In open addressing we save storage space since we don't need pointers, so we can, for the same storage cost, build larger hash tables.
Answer: True since we don't keep pointers.

(d) Collisions (i.e., that when we insert a key we don't find an empty slot) are equally probable (under same loading factor, assuming the uniform hashing assumption)
Answer: In open addressing all keys are in the hash table occupying a slot each; hence more slots are occupied on the average compared to the same keys inserted in a hash table with chaining (since in this later case more than one keys could be chained in the same slot). Then inserting a new item will find a busy slot with higher probability in the case of open addressing.

Problem 4.

(a) Consider a hash table of size $m = \sqrt{n}$. Then under the uniform hashing assumption, as $n \to \infty$, the number of empty slots tends to: i) a constant , ii) zero, iii) infinity.

(b) If $m = cn, c > 0$, then as $n \to \infty$, the number of empty slots tends to: i) a constant , ii) zero, iii) infinity.

(c) If $m = n^a, a > 1$, then as $n \to \infty$, the number of empty slots tends to: i) a constant , ii) zero, iii) infinity.

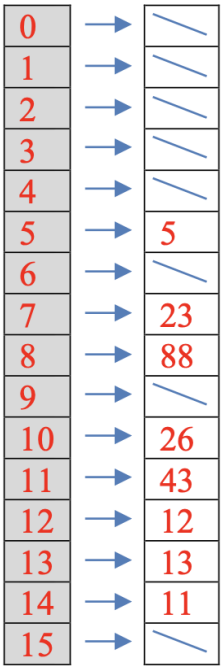
Solution

The answers are: a): ii), b): iii), c) iii). The average number of empty slots is $S = m(1-1/m)^n$. Since $n = m^2$, $S = m((1-1/m)^m)^m \to m(1/e)^m \to 0$. Similarly we can show that in the other cases $S \to \infty$.

Problem 5.

Let H be an open address hash table, initialized with 4 slots. Assume that the load factor cannot exceed 0.75, and assume that re-hashing is done by doubling. During each insertion of a new element into H, suppose that the hash function used to generate a probe sequence is obtained via linear probing, based on the auxiliary hash function $h(k) = k \mod m$, where m equals the number of slots in the hash table. Draw the final open address hash table obtained after inserting into H the following integers 12, 43, 13, 88, 23, 26, 11, 5 in this given order.

Solution



Problem 6.

Collision Resolution Assume simple uniform hashing in the entire problem.

(a) Consider a hash table with m slots that uses chaining for collision resolution. The table is initially empty. What is the probability that, after four keys are inserted, there is a chain of size exactly 3?
Solution: For the three keys that collide, the probability that they do is $\frac{1}{m^2}$, the probability that the last two hash into the same slot as the first one. The probability that the fourth key does not hash into that slot is $\frac{(m-1)}{m}$. So the probability of a chain of three for a given three keys is $\frac{(m-1)}{m^3}$, and there are $\binom{4}{3} = 4$ ways of choosing these three keys out of the four, so the total probability of a chain of exactly three is $\frac{4(m-1)}{m^3}$

(b) Consider a hash table with m slots that uses open addressing with linear probing. The table is initially empty. A key k_1 is inserted into the table, followed by key k_2, and then key k_3. What is the probability that the total number of probes while inserting these keys is at least 4?
Solution: Since the insertion of each element requires at least one probe, for there to be 4 probes, there needs to be at least one collision. If the first two keys collide, we do not even need to know where the third key hashes to – there is already a collision. The probability that the first two keys collide is $1/m$. If the first two keys do not collide, then the third key needs to hash to one of the two slots that the first two keys hash to in order for there to be a collision. The probability of this happening is the product of the probability that the first two keys do not collide and the probability that the third key goes into one of the two slots that the first two keys hash to $\frac{m-1}{m} \cdot \frac{2}{m} = \frac{2(m-1)}{m^2}$. The total probability of having at least 4 probes is $\frac{1}{m} + \frac{2(m-1)}{m^2} = \frac{3m-2}{m^2}$.

(c) Suppose you have a hash table where the load-factor a is related to the number n of elements in the table by the following formula:

$$a = 1 - \frac{1}{\sqrt{n}}$$

If you resolve collisions by open addressing, what is the expected time for an unsuccessful search in terms of n?
Solution: According to Theorem 11.6 in CLRS (2nd edition), the expected time taken by an unsuccessful search in open addressing (under the Uniform Hashing assumption) is $T(a) \leq 1 - \frac{1}{1-a}$. Plugging in $a(n) = 1 - \frac{1}{\sqrt{n}}$ we get $T(a) \leq \frac{1}{1/\sqrt{n}} = \sqrt{n}$.

Problem 7.

Amortized analysis

(a) For the case table doubling and halving, using the charging method (i.e., define the amortized cost by assigning appropriate additional cost to past operations) prove that the amortized cost

both of insertions and deletions is $O(1)$. Table doubling occurs when the number of stored elements n becomes $n = m$, in which case $m \to 2n$ with cost $\Theta(m)$. Table halving occurs when the table becomes $1/4$ full, i.e., $n = m/4$, in which case $m \to 2n$, incurring cost $\Theta(m)$. Hint: the case of insertions is proved in the slides. Use the same idea to prove the case of deletions.

Solution: We need to prove that when the table shrinks from m to $m/2$ there is at least a number am (for a fixed a) of deletions that occurred in the past and which can be charged (once only) for this $\Theta(m)$ cost. If this is the case, then the extra cost per such delete operation due to the table halving is $\leq \frac{\Theta(m)}{am} = \frac{cm}{am} = c/a$, i.e., constant. Hence the total amortized cost of such a delete operation remains also $O(1)$. We also need to prove that we charge these deletions only once due to future table halving events.

Note that these deletions are charged only once this extra cost.

Let's consider the previous doubling or halving event before this specific halving event $m \to m/2$. If it was a doubling event, since it produced a table of size m it had $m/2$ elements already. At the time of the halving, there were only $m/4$ left, hence there have been $m/4$ more deletions than insertions between these two event points. Since between these two events we have $\geq m/4$ deletions in total, we can use $a = 1/4$ and charge each deletion with less than $4c$, i.e., $O(1)$. We charge these deletions only once because they are uniquely corresponding to the halving event.

If the previous event was a halving event that produced a table of size m, then just before the table was $2m$ and had $m/2$ elements. Using the same argument, to go from $m/2$ to $m/4$ we must have at least $m/4$ deletions. This completes the proof using the same arguments.

(b) For the case of $2-3$ trees use the charging method to prove that we can use amortized costs of $O(log(n))$ for insertion and $\emptyset$ for deletions, where n is the size of the set of keys in the tree at the time of the insertion.

Solution: An insert charges nothing to the past. A delete in a tree of size n charges $O(log(n))$ the last insert in the history of the system that caused the tree to reach size n from $n-1$. Note that many times in the past there was an insert operation that may have caused $n-1 \to n$. We care for the last time $n-1 \to n$ occurred before our deletion event (the deletion caused $n \to n-1$).